Machine Learning 101 for Financial Trading

FULL CODE EDITION

FSE-DAT-3019-1.1 | Data Analytics and Computational Modelling

Sector: Financial Services | Proficiency Level 3

FACILITATOR GUIDE

Public edition - assessor keys withheld

Tertiary Infotech Academy Pte Ltd

UEN 201200696W

Version 2.3 | 22 October 2026

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Table of Contents

Version History **3**
1. About This Guide **5**
2. Preparing to Deliver **6**
3. Daily Lesson Plan **7**
Day 1 7
Day 2 8
4. Facilitation Notes by Topic **10**
Topic 1: Foundations of Quant and Algorithmic Trading: From Thorp to AI Agents 10
Topic 2: Technical Indicators, Trading Strategies and Metrics with TradingView and Alpaca 14
Topic 3: Machine Learning Models, Statistical Modelling and A/B Testing for Trading Signals 20
Topic 4: Building an End-to-End Algorithmic Trading Pipeline: Backtest, Forward Test and Performance Analysis 26
5. Assessment Administration **34**
6. Assessor's Keys **35**

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Version History

Version No	Date Effected	Change Update	Approved by
2.3	22 October 2026	Assessment papers: every Practical Performance sub-question now has its own answer box, with screenshot boxes naming the lab where the step was practised, and the Written Assessment's ruled lines are replaced by answer boxes, so candidates type or paste their answers into the paper and upload it to the LMS. The PP no longer names the colleague's fault in its evidence list or the program settings, and the Assessment Plan's Annex A prints the K-code on every question.	Dr Alfred Ang
2.2	22 October 2026	Screenshots added to Labs 7, 8 and 9: a Bollinger Band mean-reversion strategy in TradingView with the Paper Trading order panel (Lab 7), the simulator's paper order ticket and its passed pre-trade risk checks and fill (Lab 8), and Performance Analytics comparing a strategy with SPY buy-and-hold (Lab 9). Slide references in the Lesson Plan, Learner Guide and Facilitator Guide updated to the new slide numbers.	Dr Alfred Ang
2.1	21 October 2026	Low-code delivery. Every lab now runs in the Trading Simulator without coding, and the Practical Performance is done by configuring and running a provided trading program, with the same scenario, tasks, ability coverage and marking. A Prompt Library provides one AI prompt per lab, with one-click links, for learners who want code generated for them. Getting Started screenshots added (simulator Pipeline Map and Integrations, Alpaca paper dashboard, TradingView chart). Version History added to every document. Topic 2 slides added on mean reversion (the simulator's rules, mean reversion versus trend following, the main risk and safeguards), on the RSI look-back period (RSI(2) versus RSI(14)) and on long and short positions.	Dr Alfred Ang
2.0	18 October 2026	Major revision, focused entirely on quant and algorithmic stock trading. Courseware rebuilt and re-platformed to Alpaca (API-first broker, paper trading, 24/5 US stock trading) and TradingView (charting, Pine Script v6, Strategy Tester, Paper Trading) with Python. Four topics: quant success stories and the evolution of quant from Thorp and Renaissance through statistical arbitrage to high-frequency trading, the paradigm shift from machine learning to LLMs and its impact on quant, and AI agents versus agentic AI in trading; the classification of trading algorithms, indicators, strategies, a TradingView versus Alpaca comparison, order placement and metric definitions; machine learning models, statistical modelling, A/B testing and pairs trading as a hedging algorithm; and an end-to-end, automated algorithmic trading pipeline. 'How the algorithm works' slides explain each algorithm on real market data. Twenty hands-on labs build one pipeline stage by stage, each with an instruction PDF containing step-by-step AI prompts, and include live paper trading of a strategy. Assessment: Written Assessment (short-answer questions, K1-K9) and Practical Performance (four tasks, A1-A10) with a standalone SMART marking rubric. Facilitator Guide added. Fee restated in SGD exclusive of GST, optimum class size stated, provider name updated to Tertiary Infotech Academy Pte Ltd.	Dr Alfred Ang

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Version No	Date Effected	Change Update	Approved by
1.1	22 December 2022	Multiple choice questions removed; practical test retained as the sole assessment instrument.	Dr Alfred Ang
1.0	10 June 2022	Initial release for pilot launch, aligned to Data Analytics and Computational Modelling (Proficiency Level 3).	Dr Alfred Ang

This document is at Version 2.3, effective 22 October 2026.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

1. About This Guide

This public edition of the Facilitator Guide omits the assessor's keys. This Facilitator Guide supports the trainer and assessor in delivering Machine Learning 101 for Financial Trading. It sets out the course information, the daily lesson plan with slide references, facilitation notes and timing cues for every topic and lab, the administration of the assessment and its timings. The assessor's keys, model answers and rubric are held in the trainer-only full edition and the Assessment Plan.

Item	Details
Course Title	Machine Learning 101 for Financial Trading
TSC Code and Title	FSE-DAT-3019-1.1 Data Analytics and Computational Modelling
Proficiency Level	3 - Identify and utilise appropriate statistical algorithms and data models to perform data analyses, test hypotheses and derive inferences, patterns or solutions from data
Course Duration	16 hours over 2 training days (14 hours classroom training + 2 hours assessment)
Daily Schedule	0900 - 1830 (8 instructional hours per day, excluding two 15-minute tea breaks and a 1-hour lunch break)
Mode of Delivery	Instructor-led classroom training with hands-on practical laboratory exercises on the Alpaca paper-trading platform, TradingView and Python
Class Size / Ratio	Optimum class size of 20 participants (maximum 20) | Trainer : learner 1 : 20
Assessment	Written Assessment (45 min, K1-K9) | Practical Performance (75 min, A1-A10)
Funding	IBF Standards Training Scheme (IBF-STS), subject to eligibility; at least 75% attendance and passing the assessment are required
Companion documents	Lesson Plan, Trainer Slides, Learner Guide, Assessment Plan, Assessment Rubric, lab folder (labs)

Learning outcomes

LO	Learning Outcome	TSC Ref
LO1	Develop hypotheses for implementing data analytics and computational modelling activities in algorithmic trading, and identify possible big data applications across the forms of data within financial services.	A1, A6, K1, K9
LO2	Identify appropriate statistical algorithms and data models to test trading hypotheses, and use appropriate analytics platforms, analytical tools and statistical methods given specific analytics and reporting requirements.	A2, A3, K3, K5, K7
LO3	Conduct statistical modelling of market data with machine learning models to derive patterns and solutions, and perform A/B testing of trading strategies based on different hypotheses.	A4, A5, K2, K4, K6
LO4	Perform coding and configuration of an automated trading program based on selected models or algorithms, conduct tests on its actions and outcomes, diagnose unintended outcomes, and draw insights from the results to support decisions and report accordingly.	A7, A8, A9, A10, K7, K8

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

2. Preparing to Deliver

Two weeks before the class

- Send the joining instructions: bring a laptop with Python 3.10+, create a free Alpaca account and a free TradingView account in advance if possible.
- Confirm the venue network allows outbound HTTPS to paper-api.alpaca.markets, data.alpaca.markets and tradingview.com. If it does not, plan to run every lab in offline mode - the bundled data and simulator make this seamless.
- Refresh the bundled data if required and re-run every lab script once on the trainer laptop (setup steps are in the Lab 1 instruction PDF, labs_fullcode/Lab01-alpaca-setup/Lab01_Instructions.pdf). Each lab folder holds its LabNN_Instructions.pdf with steps, AI prompts and checks.

On the day

- Copy the lab folder to each learner (USB or LMS download) and check that pip install -r requirements.txt completes before Lab 1.
- Log in to the trainer Alpaca paper account and TradingView for live demonstrations; never use a live-money account.
- Have the Learner Guide open at the setup section; most first-hour issues are Python environment issues, not course content.
- Prepare the attendance sheet for each session and the assessment attendance record.

Safety and conduct messages to repeat

- Paper trading only; no real money at any point.
- Lion City Capital and every scenario in these materials are fictitious and for training only. All trading in this course is paper (simulated) trading. Nothing in these materials is investment advice or a recommendation to trade any security.
- API keys are credentials: never put them on a slide, in a chat, or in a file that is shared.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

3. Daily Lesson Plan

Identical to the Lesson Plan; the Slides column is read from the built deck.

Day 1

Time	Duration	Topic / Activity	LO	TSC Ref	Slides
09:00 - 09:30	30 min	Course Overview, Trainer Introduction and Ground Rules	-	-	1-34
09:30 - 10:45	75 min	T1.1-1.4 Quant Success Stories, the Evolution of Quant, the ML-to-LLM Shift, and Agentic AI versus AI Agents	LO1	K1, K9	36-58
10:45 - 11:00	Tea Break				
11:00 - 11:30	30 min	T1.5-1.6 Forms of Financial Data, Big Data Applications and Trading Hypotheses	LO1	A1, A6, K1, K9	59-66
11:30 - 11:50	20 min	Lab 1: Pipeline Skeleton, Configuration and the Alpaca Paper Account	LO1	K5	67
11:50 - 12:10	20 min	Lab 2: Data-Validation Stage: Cross-Check Pipeline Bars against TradingView	LO1	A3, K5, K9	68
12:10 - 12:30	20 min	Lab 3: Ingestion Stage: Multi-Timeframe Bars, Headlines and Data Quality	LO1	A6, K9	70
12:30 - 13:30	Lunch Break				
13:30 - 13:45	15 min	Lab 4: Hypothesis Stage: from a Written Card to an Automated Statistical Test	LO1	A1, A6, K1	73
13:45 - 14:30	45 min	T2.1-2.2 Technical Indicators and Standard Algorithmic Trading Strategies	LO2	A2, K3	77-89, 95-117
14:30 - 14:55	25 min	Lab 5: Indicator Stage: Pine Script v6 Indicators and their Python Twins	LO2	A3, K3, K7	123
14:55 - 15:25	30 min	Lab 6: Strategy Prototype: TradingView Strategy Tester, Paper Trading from the Chart and Pipeline Reconciliation	LO2	A2, A3, K3, K7	126
15:25 - 15:40	Tea Break				
15:40 - 16:05	25 min	Lab 7: Signal Stage: a Library of Trend, Mean-Reversion, Momentum and Breakout Signals	LO2	A2, K3, K7	130
16:05 - 16:50	45 min	T2.3-2.4 Analytics Platforms, Coding Languages and Order Placement	LO2	A3, K5, K7	90-91, 118-122
16:50 - 17:20	30 min	Lab 8: Execution Stage: an Order Manager that Trades the Paper Account in Any Session	LO2	A3, K5	133
17:20 - 17:50	30 min	T2.5 Performance and Risk Metric Definitions	LO2	A3, K3	92-94
17:50 - 18:15	25 min	Lab 9: Evaluation Stage: Performance and Risk Metrics with a Benchmark Table	LO2	A3, K3	136
18:15 - 18:30	15 min	Day 1 Recap and Q&A	-	-	75, 139

Instructional time Day 1: 480 minutes.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Day 2

Time	Duration	Topic / Activity	LO	TSC Ref	Slides
09:00 - 09:15	15 min	Day 1 Recap and Day 2 Overview	-	-	75, 139
09:15 - 09:40	25 min	T3.1-3.2 Statistical Analyses, Algorithms and Supervised Learning Models	LO3	A4, K2, K4	142-144, 148-150, 155
09:40 - 10:00	20 min	Lab 10: Strategy-Search Stage: Grid Search, Out-of-Sample Ranking and Decay	LO3	A2, A4, K2	159
10:00 - 10:20	20 min	Lab 11: ML-Signal Stage: a Walk-Forward Classifier from Probability to Position	LO3	A4, K4, K6	161
10:20 - 10:35	Tea Break				
10:35 - 10:55	20 min	T3.3-3.4 Unsupervised Learning and Statistical Modelling Techniques	LO3	A4, K4, K6	145-146, 151-153
10:55 - 11:10	15 min	Lab 12: Regime-Filter Stage: PCA and K-Means as a Gate on the Signals	LO3	A4, K4, K6	163
11:10 - 11:25	15 min	T3.5-3.6 A/B Testing, and Pairs Trading as a Hedging Algorithm	LO3	A4, A5, K2, K6	147, 154, 156-158
11:25 - 11:40	15 min	Lab 13: A/B-Test Stage: a Statistical Decision before Capital Moves	LO3	A5, K2	165
11:40 - 12:00	20 min	Lab 14: Pairs Stage: a Hedged, Market-Neutral Statistical-Arbitrage Sleeve	LO3	A4, K6	167
12:00 - 12:30	30 min	T4.1-4.2 Pipeline Architecture and Backtesting with Trading Costs	LO4	A7, A8, K7	171-174, 177
12:30 - 13:30	Lunch Break				
13:30 - 13:55	25 min	Lab 15: Backtest-Engine Stage: Cost Models, Execution Timing and Break-Even	LO4	A7, A8, K7	182
13:55 - 14:05	10 min	T4.3 Forward Testing with Automated Paper Trading	LO4	A7, A8	178-179
14:05 - 14:30	25 min	Lab 16: Automation Stage: a Scheduled, Risk-Checked Forward Test and Live Paper Trading	LO4	A7, A8, K7	184
14:30 - 14:50	20 min	Lab 17: Event-Driven Automation: TradingView Webhook to Signal, Risk and Execution	LO4	A7, K5, K7	186
14:50 - 15:00	10 min	T4.4 Potential Reasons for Unintended Outcomes	LO4	A9, K8	175, 180
15:00 - 15:20	20 min	Lab 18: Diagnostics Stage: Automated Bias Checks before Go-Live	LO4	A9, K8	187
15:20 - 15:35	Tea Break				
15:35 - 15:45	10 min	T4.5 Performance Analysis, Insights and Reporting	LO4	A10	176
15:45 - 16:10	25 min	Lab 19: Monitoring and Reporting Stage: Automated Tearsheet and Backtest-vs-Forward Report	LO4	A8, A10	189
16:10 - 16:30	20 min	Lab 20: Capstone: One Command Runs the Pipeline End to End, with an Agentic Layer and Human Approval	LO4	A7, A10, K1	191
16:30 - 17:15	45 min	Written Assessment (Short Answer Questions)	All LOs	K1 - K9	23
17:15 - 18:30	75 min	Practical Performance Assessment	All LOs	A1 - A10	23

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Instructional time Day 2: 480 minutes.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

4. Facilitation Notes by Topic

Topic 1: Foundations of Quant and Algorithmic Trading: From Thorp to AI Agents

Day 1 | Slides 35-75 | A1, A6, K1, K9 | Labs 1-4

Learning outcome: Develop hypotheses for implementing data analytics and computational modelling activities in algorithmic trading, and identify possible big data applications across the forms of data within financial services.

Key teaching points

- 1.1 Quant Success Stories: What Systematic Trading Has Achieved (K1): Ed Thorp - from blackjack to markets: his fund (Convertible Hedge Associates, 1969; Princeton Newport Partners from 1974) is often called the first market-neutral hedge fund and earned about 20% a year after fees for two decades by hedging warrants and convertibles with probability models.
- 1.2 The Evolution of Quant: From Thorp and Renaissance to High-Frequency Trading (K1, K9): Algorithmic trading is the use of a computer program that follows a defined set of instructions to generate and execute orders automatically, based on price, timing, volume or a mathematical model.
- 1.3 The Paradigm Shift from Machine Learning to LLMs, and Its Impact on Quant (K1, K9): Classical machine-learning quant works on structured data - prices, volumes, fundamentals - with features engineered by the quant (lagged returns, RSI, factor scores) and models such as regression, random forests and gradient boosting fitted to labelled history.
- 1.4 Agentic AI versus AI Agents in Trading, and Multi-Agent Trading Systems (K1): Agentic AI is the capability of AI to plan and act: it calls tools (market-data and broker APIs), uses skills and memory, and follows instructions and guardrails to complete multi-step work when it is asked - for example, an LLM that fetches a stock's bars and news, checks its memory of past trades and proposes an order within its instructions.
- 1.5 Forms of Data within Financial Services and Big Data Applications in Trading (K9, A6): Market data: OHLCV bars (open, high, low, close, volume) at timeframes from one minute to one month; quotes (best bid and ask); trades (every print); and order-book depth. Alpaca's Market Data API serves bars, quotes, trades and news.
- 1.6 Developing Hypotheses for Quant and Algorithmic Trading Models (A1, K1): Every trading model starts with a hypothesis: a precise, testable statement about market behaviour - not 'buy good stocks' but 'when SPY closes below its lower Bollinger Band, its average 5-day forward return exceeds its unconditional 5-day return'.

Facilitation cues

- Open with the success stories (Thorp, Renaissance, D.E. Shaw, Man AHL, Two Sigma, the quant market makers) and ask: 'What did they all have in common?' - a testable hypothesis, clean data, out-of-sample testing, costs and risk control.
- Walk the evolution of quant as a chain: probability and option pricing -> factor models -> statistical arbitrage -> HFT speed race -> ML -> LLMs and agents. Tie each failure to a control used later: LTCM -> leverage limits; quant quake -> crowding; Knight Capital -> deployment testing and the kill switch.
- Define agentic AI as the capability (tools, skills, memory, instructions) and an AI agent as its special always-on class that stands by 24/7 (OpenClaw, Hermes Agent, OpenAI Dots).
- On the ML-to-LLM shift, stress the new backtest risk: an LLM may 'know' its training period, so LLM signals must be tested after the knowledge cutoff.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- Insist that every hypothesis in Lab 4 states all five elements before any data is examined. This is assessed in PP Task 1.

Lab 1: Pipeline Skeleton, Configuration and the Alpaca Paper Account

Item	Facilitation
TSC / duration / slide	K5 | 20 min | slide 67
Folder and files	labs_fullcode/Lab01-alpaca-setup - Lab01-alpaca-setup/check_stage.py, pipeline/config.yaml, pipeline/config.py, pipeline/clock.py, pipeline/check.py, pipeline/__main__.py, Lab01-alpaca-setup/solution/, .env.example, requirements.txt
Objective	Create the pipeline package and its config.yaml (universe, strategy parameters, risk limits, costs, secret names), build a validating configuration loader and a market-clock module, and run `python -m pipeline check` against the Alpaca paper account or the offline simulator.
Run	Run Lab 01: python3 Lab01-alpaca-setup/check_stage.py (same as: python -m pipeline check) The complete code: pipeline/check.py, pipeline/clock.py, pipeline/config.py (reference copy: Lab01-alpaca-setup/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Set up Python and your AI coding assistant (K5) Store Alpaca PAPER keys outside the code (K5) Write the pipeline configuration (K5) Build the configuration loader with safety validation (K5) Add the market clock for US and Singapore time (K5) Build the check command and the pipeline CLI (K5) Run the pre-flight check (K5) Prove the safety rules work (K5)
Expected output	python -m pipeline check --reset (offline): configuration valid; 8 libraries OK; 4 secrets 'not set'; account SIM-LCC-0001 ACTIVE with equity, cash and buying power USD 100,000.00 and 0 positions; a clock line in New York and Singapore time (CLOSED at weekends, OVERNIGHT or PRE during a weekday Singapore class); last SPY bar 2025-12-31, close 676.64; 'RESULT: PIPELINE READY - paper trading only', exit code 0. python -m pipeline lists 24 commands.
Common errors	ModuleNotFoundError: pipeline, common or yaml -> Activate the venv, run pip install -r requirements.txt and run every command from inside labs. 401 Unauthorized or forbidden online -> The keys are wrong or are LIVE keys. Regenerate PAPER keys and check .env has no spaces or quotes around the values. Mode shows OFFLINE although keys are set -> Remove --offline, set mode: auto in config.yaml and make sure LAB_OFFLINE is not 1 in .env. The assistant wrote code that prints the secret -> Reject the change: only the first 4 characters of the key ID may ever be shown, never the secret.
Debrief prompt	A trading pipeline is only as safe as its configuration and its connection to the broker. One validated config file, paper-only enforcement, keys held in the environment and a pre-flight check that a scheduler can read from the exit code show how AI and algorithmic trading programs are connected to execution platforms in a controlled way (K5).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 2: Data-Validation Stage: Cross-Check Pipeline Bars against TradingView

Item	Facilitation
TSC / duration / slide	A3, K5, K9 | 20 min | slide 68
Folder and files	labs_fullcode/Lab02-tradingview-charts - Lab02-tradingview-charts/validate_stage.py, Lab02-tradingview-charts/lcc_data_check.pine, Lab02-tradingview-charts/tradingview_values.csv, pipeline/validate.py, pipeline/reference/AAPL_1Day_raw.csv, pipeline/reference/D05.SI_1Day_raw.csv, Lab02-tradingview-charts/solution/validate.py
Objective	Read daily bars for SGX:D05 and NASDAQ:AAPL from the TradingView Data Window, record them, and build a validation stage that compares TradingView with the pipeline's raw and dividend-adjusted bars and writes a validation report.
Run	Run Lab 02: python3 Lab02-tradingview-charts/validate_stage.py (same as: python -m pipeline validate) The complete code: pipeline/validate.py (reference copy: Lab02-tradingview-charts/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Add the data-check panel to SGX:D05 (A3, K9) Repeat for NASDAQ:AAPL and switch the adjustment (A3, K9) Record what TradingView shows (A3) Build the validation stage (A3, K5, K9) Run the validation stage (A3, K9) Understand adjusted versus unadjusted prices (K9) Make the validation fail on purpose (A3) Identify other forms of market data (K9)
Expected output	python -m pipeline validate --date 2025-12-31: AAPL RAW close 271.86 vs ADJUSTED 271.12 (factor 0.9973); SGX:D05 RAW close 56.36 vs ADJUSTED 54.22 (factor 0.9620); both match TradingView's default (unadjusted) view to the cent - 'PASS - TradingView matches RAW'; RESULT: 2/2 watch-list symbols validated; files _outputs/Lab02_validation_report.csv and _outputs/Lab02_validation.png.
Common errors	SGX:D05 shows delayed or no intraday data -> Free plans show delayed SGX data; daily bars are enough for this lab. Verdict FAIL for D05 -> Check the date, that the chart is 1D and that 'Adjust data for dividends' is unticked; enter the regular-session close, not an extended-hours price. Data Window not visible -> Open it from the right toolbar (Data Window icon) or press Alt+D. Pine compile error -> The first line must be //@version=6 and the whole file must be pasted.
Debrief prompt	Charting platforms are analytics tools (A3) and the API is the pipeline's data path (K5); a validation stage proves they agree before anything is automated. Knowing that adjusted and unadjusted prices are different forms of the same data (K9) prevents a classic error: training on adjusted history and then trading on raw quotes without converting.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 3: Ingestion Stage: Multi-Timeframe Bars, Headlines and Data Quality

Item	Facilitation
TSC / duration / slide	A6, K9 | 20 min | slide 70
Folder and files	labs_fullcode/Lab03-alpaca-market-data - Lab03-alpaca-market-data/ingest_stage.py, pipeline/ingest.py, pipeline/reference/sample_headlines.json, Lab03-alpaca-market-data/solution/ingest.py
Objective	Build `python -m pipeline ingest` to fetch and cache daily, weekly, monthly and hourly bars for the universe, run automated data-quality checks, ingest headlines as unstructured data, and describe the forms of data and the 3 Vs of big data.
Run	Run Lab 03: python3 Lab03-alpaca-market-data/ingest_stage.py (same as: python -m pipeline ingest) The complete code: pipeline/ingest.py (reference copy: Lab03-alpaca-market-data/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Understand how bars are requested (K9) Build the cache and load_bars() (A6, K9) Add automated data-quality checks (A6) Ingest an unstructured source: news headlines (A6, K9) Build the ingest run with the report (A6, K9) Run the ingestion stage (A6, K9) Inspect the cache and the manifest (A6) Design a big-data extension (A6)
Expected output	python -m pipeline ingest (offline): 37 files cached in _cache/bars, 0 failed quality checks; daily 2,011 bars per symbol, weekly 418, monthly 96, SPY hourly 440 (Q4 2025); big-move flags NVDA 40, AAPL 12, SPY 5; 12 synthetic headlines cached; SPY annualised mean 15.14% and volatility 19.46% (excess kurtosis 13.12), AAPL 28.66% / 30.79%, correlation 0.77; files _outputs/Lab03_data_quality.csv, _cache/manifest.json and chart _outputs/Lab03_market_data.png.
Common errors	Hourly bars 'not available offline' for most symbols -> Expected: only SPY hourly data are bundled; online Alpaca returns hourly bars for every symbol. Alpaca returns fewer hourly bars than expected -> The free IEX feed covers only IEX trades; use daily bars or a SIP subscription for full volume. Status CHECK on many symbols -> CHECK means 'look at the big moves' (earnings, splits, 2020 crash); only FAIL blocks a file. Later stages ignore new data -> load_bars reads the cache offline; rerun python -m pipeline ingest after adding data.
Debrief prompt	Ingestion is the first automated stage: it fixes one version of the truth for every later stage and blocks bad data before a model can learn from it. Bars, quotes, trades, news and alternative data are different forms of financial data (K9) with different volume, velocity and variety - which is what turns a trading data set into a big-data application (A6).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 4: Hypothesis Stage: from a Written Card to an Automated Statistical Test

Item	Facilitation
TSC / duration / slide	A1, A6, K1 | 15 min | slide 73
Folder and files	labs_fullcode/Lab04-trading-hypothesis - Lab04-trading-hypothesis/hypothesis_stage.py, pipeline/hypotheses/spy_dip_after_streak.yaml, pipeline/hypothesis.py, Lab04-trading-hypothesis/solution/hypothesis.py, Lab04-trading-hypothesis/solution/spy_dip_after_streak.yaml
Objective	Write a falsifiable hypothesis card in YAML (H1, H0, signal known at the close, outcome, test, decision rule, big-data use case) and build `python -m pipeline hypothesis` to test it with a Welch t-test, a bootstrap confidence interval and a robustness grid.
Run	Run Lab 04: python3 Lab04-trading-hypothesis/hypothesis_stage.py (same as: python -m pipeline hypothesis) The complete code: pipeline/hypothesis.py (reference copy: Lab04-trading-hypothesis/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Read the hypothesis card (A1, K1) Write a card for your own idea (A1, K1) Build the test engine (A1) Add the robustness grid, chart and result file (A1, A6) Run the test on the desk's card (A1) Read the robustness grid (A1, K1) Study the shape of the effect (A1) Test your own card and its big-data use case (A1, A6)
Expected output	python -m pipeline hypothesis: SPY streak >= 3 - 178 signal days (8.9%), mean next-day return +0.2828% vs +0.0384% (all days +0.0601%), hit rate 56.7% vs 55.4%, bootstrap CI [+0.0531%, +0.5136%], Welch t = 2.03, p = 0.044, edge +24.4 bps -> SUPPORTED; robustness 3 of 6 variants supported, Bonferroni p < 0.0083; files _outputs/Lab04_hypothesis_result.yaml and chart _outputs/Lab04_hypothesis.png.
Common errors	p-value changes when you change the dates -> Expected - small samples are sensitive. Record it as evidence, not as a problem to hide. ValueError: unknown signal type -> Use losing_streak, rsi_below or close_below_sma, or ask your assistant to add a new type to condition(). Very few signal days -> With fewer than about 30 observations the test has little power; loosen the condition. Unsure what H0 should be -> H0 is 'no difference': the mean forward return on signal days equals the mean on other days.
Debrief prompt	Writing the hypothesis before testing, with a null, a cost-aware decision rule and a count of the variants tried, is the foundation of data-driven trading (A1). AI and machine-learning methods (K1) only add value when they test well-posed questions, and larger data sets (A6) are how a promising effect in one ETF is confirmed or rejected.

Topic 2: Technical Indicators, Trading Strategies and Metrics with TradingView and Alpaca

Day 1 | Slides 76-140 | A2, A3, K3, K5, K7 | Labs 5-9

Learning outcome: Identify appropriate statistical algorithms and data models to test trading hypotheses, and use appropriate analytics platforms, analytical tools and statistical methods given specific analytics and reporting requirements.

Key teaching points

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- 2.1 Technical Indicators: Trend, Momentum, Volatility and Volume (K3): Simple Moving Average (SMA): the average close over n bars. Exponential Moving Average (EMA): a weighted average with smoothing factor 2/(n+1), reacting faster to recent prices. Both are trend indicators.
- 2.2 Classification of Algorithmic Trading and Quant Algorithms (K3, A2): By purpose, trading algorithms fall into seven families: execution, market making, arbitrage, directional (alpha), statistical arbitrage and pairs (hedged), factor and portfolio construction, and hedging and risk control.
- 2.3 Analytics Platforms and Tools: TradingView versus Alpaca, and Python (K5, A3): TradingView is a charting and strategy-design platform: global markets including SGX, built-in indicators, the Pine Script language, the Strategy Tester for backtests, alerts and webhooks (webhooks need a paid plan), and a built-in Paper Trading account opened from the chart's Trading Panel with your own starting capital and commission.
- 2.4 Coding Languages for Algorithms and Signals, and Order Placement (K7): Pine Script v6 (TradingView's language): every script starts with //@version=6 and declares indicator() or strategy(). Built-in functions such as ta.ema, ta.rsi, ta.crossover and strategy.entry express a signal in a few lines.
- 2.5 Performance and Risk Metric Definitions (K3, A3): Return metrics: total return; compound annual growth rate CAGR = (ending value / starting value)^(1/years) - 1; and the benchmark-relative excess return.

Facilitation cues

- Demonstrate the Pine Editor live: paste the Lab 5 script, add it to the chart, then change one input and show the chart update.
- When explaining metrics, compute one Sharpe ratio by hand on the whiteboard from five daily returns before learners run Lab 9.
- Emphasise order status in Lab 8: a submitted order is not a filled order.
- On the RSI look-back slides, show SPY on 10 October 2025 in the simulator chart with the lower pane on RSI: RSI(2) fell to 5.8 while RSI(14) stayed near 42 - same day, opposite signals, because only the look-back differs.
- Before the strategy families, check that every learner can say what a short position is and why its loss is unlimited; then, on the mean-reversion risk slide, ask which safeguard would have limited the March 2020 losses.
- Learners who finish early: ask them to add an RSI filter to the Lab 6 strategy and compare the Strategy Tester metrics.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 5: Indicator Stage: Pine Script v6 Indicators and their Python Twins

Item	Facilitation
TSC / duration / slide	A3, K3, K7 | 25 min | slide 123
Folder and files	labs_fullcode/Lab05-pine-indicators - Lab05-pine-indicators/indicators_stage.py, Lab05-pine-indicators/lcc_trend_bands.pine, Lab05-pine-indicators/lcc_rsi.pine, Lab05-pine-indicators/tradingview_indicator_values.csv, pipeline/indicators.py, Lab05-pine-indicators/solution/indicators.py
Objective	Add the desk's Pine Script v6 indicators to a TradingView chart, build pipeline/indicators.py with twins of ta.ema, ta.sma, ta.rsi, ta.bb, ta.atr, ta.macd, ta.crossover and ta.crossunder, and cross-check the values against the TradingView Data Window.
Run	Run Lab 05: python3 Lab05-pine-indicators/indicators_stage.py (same as: python -m pipeline indicators) The complete code: pipeline/indicators.py (reference copy: Lab05-pine-indicators/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Add the trend and bands indicator (A3, K3) Add the RSI pane (A3, K3) Record the TradingView values (A3) Build the Pine twins (K3) Build the cross-check run (A3, K3) Run the cross-check (A3, K3) Change the inputs and count the extra trading (K3, K7) Create the alert the automation stage will use (A3, K7)
Expected output	python -m pipeline indicators --symbol AAPL --date 2025-12-31: adjusted close 271.12, EMA 20/50 273.23 / 268.80, Bollinger 281.58 / 274.72 / 267.87, RSI(14) 45.69; raw (TradingView default) 271.86, 273.98 / 269.60, 282.35 / 275.47 / 268.60, RSI 45.64; largest gap to TradingView 0.006% -> PASS; 2 bullish and 2 bearish EMA crosses since June 2024; files _outputs/Lab05_AAPL_indicators.csv and chart _outputs/Lab05_pine_preview.png.
Common errors	Compilation error in the Pine Editor -> The first line must be //@version=6 and the whole file must be pasted. Indicator draws in a separate pane -> lcc_trend_bands.pine must keep overlay = true; only lcc_rsi.pine uses overlay = false. Gap to TradingView above 0.5% -> Check the chart is 1D, 'Adjust data for dividends' is unticked, and you read the 2025-12-31 bar; compare with the RAW column. FileNotFoundError for the RAW column -> Raw reference files exist for AAPL and D05.SI only; other symbols show the adjusted column.
Debrief prompt	Technical indicators (K3) turn prices into trend, volatility and momentum readings; defining them once in Pine Script and once in Python, and proving they match (A3), keeps traders and the automated pipeline on the same numbers. Faster settings fire more signals, which means more trades and more cost (K7).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 6: Strategy Prototype: TradingView Strategy Tester, Paper Trading from the Chart and Pipeline Reconciliation

Item	Facilitation
TSC / duration / slide	A2, A3, K3, K7 | 30 min | slide 126
Folder and files	labs_fullcode/Lab06-pine-strategy-tester - Lab06-pine-strategy-tester/reconcile_stage.py, Lab06-pine-strategy-tester/lcc_ma_crossover_strategy.pine, Lab06-pine-strategy-tester/tv_strategy_results.yaml, pipeline/reconcile.py, Lab06-pine-strategy-tester/solution/reconcile.py
Objective	Configure a Pine Script v6 strategy with costs, read the Strategy Tester, build `python -m pipeline reconcile` to check the Pine settings and reproduce the results, and place and manage paper trades from the chart with TradingView Paper Trading (and optionally an Alpaca paper account).
Run	Run Lab 06: python3 Lab06-pine-strategy-tester/reconcile_stage.py (same as: python -m pipeline reconcile) The complete code: pipeline/reconcile.py (reference copy: Lab06-pine-strategy-tester/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Read the strategy code (A2, K3) Run it in the Strategy Tester and record the results (A3, K7) Build the reconciliation stage (A2, A3, K3) Reconcile TradingView with the pipeline (A2, A3) Explain the differences (A3) Stress the costs and the speed of the rule (K7) Put the strategy into action with TradingView Paper Trading (A3) Manage the paper position from the chart (A3, K7) Optional - connect the Alpaca paper account in TradingView (A3) Decide whether to promote the rule (A2)
Expected output	python -m pipeline reconcile --symbol SPY --fast 50 --slow 200: 6 settings checks PASS; net profit USD 79,878 (+79.88%), 4 closed trades + 1 open, 50.0% profitable, profit factor 5.03, max drawdown -33.72%, Sharpe 0.55, Sortino 0.76, commission USD 9.00 versus buy and hold +187.26%; SMA 20/50: +88.23%, 16 trades, commission USD 33; TradingView Paper Trading shows a filled market order, a working limit order and a bracket; files _outputs/Lab06_reconciliation.yaml and chart _outputs/Lab06_crosscheck.png.
Common errors	Strategy Tester shows 'No data' -> Check the backtest dates in the inputs and that the chart is 1D. Different first trade -> Expected: the bundled data starts in 2018, so the pipeline's SMA 200 exists only from October 2018. A settings check FAILs -> Someone changed the Pine file or config.yaml costs.tradingview; make them agree and rerun. Paper Trading not listed -> Open the Trading Panel (bottom toolbar) and scroll the broker list; it is called 'Paper Trading by TradingView'.
Debrief prompt	Choosing a rule and its parameters is a modelling decision (A2); the Strategy Tester and TradingView Paper Trading are analytics and execution platforms (A3) that report standard metrics (K3). An independent reconciliation is the gate between a chart prototype and the automated pipeline, and costs typed into strategy() (K7) barely touch a slow rule but compound for a fast one.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 7: Signal Stage: a Library of Trend, Mean-Reversion, Momentum and Breakout Signals

Item	Facilitation
TSC / duration / slide	A2, K3, K7 | 25 min | slide 130
Folder and files	labs_fullcode/Lab07-technical-indicators - Lab07-technical-indicators/signals_stage.py, pipeline/signals.py, Lab07-technical-indicators/solution/signals.py
Objective	Build pipeline/signals.py with trend, mean-reversion, momentum and breakout signals that return target positions with next-bar alignment, a registry, an automated look-ahead test and a universe target file for the execution stage.
Run	Run Lab 07: python3 Lab07-technical-indicators/signals_stage.py (same as: python -m pipeline signals) The complete code: pipeline/signals.py (reference copy: Lab07-technical-indicators/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Agree the signal contract (A2) Build the trend signals (A2, K3) Build mean-reversion, momentum and breakout signals (A2, K3) Add the registry, next-bar alignment and the look-ahead test (A2, K3) Build the signals command (A2) Run the signal stage (A2, K3) Catch a look-ahead bug on purpose (K3) Relate signal frequency to trading cost (K7)
Expected output	python -m pipeline signals --symbol SPY: eight signals in four families, all pass the look-ahead truncation test; ema_cross 26 entries and 71% exposure (latest flip at the close of 2025-11-28, held from 2025-12-01); dip_buy 102 entries; universe targets 7 of 9 long (MSFT, TLT flat) saved to _outputs/Lab07_targets.csv; chart _outputs/Lab07_signals.png.
Common errors	Signals contain NaN -> Wrap conditions with _flat_until_ready so warm-up bars are 0, not NaN. A look-ahead test fails -> Look for shift(-1), center=True or any use of data after bar t in that signal. Targets file is empty -> Run python -m pipeline ingest first or check universe.trade in config.yaml. TypeError on a column called hist -> Use df['hist'], not df.hist - hist is also a pandas method.
Debrief prompt	A single signal contract - target positions decided at the close, held from the next bar - is what lets one library feed backtests, live execution and diagnostics (A2). Indicators (K3) become signals through simple, testable rules, and the number of entries each rule produces is the first estimate of its trading cost (K7).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 8: Execution Stage: an Order Manager that Trades the Paper Account in Any Session

Item	Facilitation
TSC / duration / slide	A3, K5 | 30 min | slide 133
Folder and files	labs_fullcode/Lab08-paper-orders - Lab08-paper-orders/execution_stage.py, pipeline/execution.py, pipeline/clock.py, common/broker.py, Lab08-paper-orders/solution/execution.py
Objective	Build pipeline/execution.py: a session-aware, idempotent order manager that sizes orders from target weights, sends market, extended-hours limit, GTC-queued or bracket PAPER orders, polls their lifecycle, cancels protective legs before exits, and falls back safely when an order is rejected.
Run	Run Lab 08: python3 Lab08-paper-orders/execution_stage.py (same as: python -m pipeline execute) The complete code: pipeline/execution.py (reference copy: Lab08-paper-orders/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Learn the order rules for each session (K5) Build the planning logic: target vs actual (A3) Build session-aware, idempotent order submission (A3, K5) Add polling, protective-order handling and safe fallbacks (A3, K5) Run the order manager in the current session (A3, K5) Compare the sessions with the teaching override (K5) Bracket orders and a clean exit (A3, K5) Trade LIVE on the Alpaca paper account during class (A3, K5) Optional - watch a fill right now with the crypto demo (K5) Flatten the live paper account (A3)
Expected output	python -m pipeline execute --reset --session regular (offline): BUY 44 SPY and BUY 36 AAPL as MARKET DAY orders, filled at 676.70 and 271.15, equity USD 99,996.05; the idempotency re-run sends no new order and a duplicate submission is skipped; --session overnight sends LIMIT DAY extended_hours orders (filled 676.63 / 271.12); --session closed queues LIMIT GTC orders (accepted); bracket legs SPY 710.47 / 656.34; --flatten cancels the protective legs and exits (equity USD 99,992.09). With --live-paper the same flow runs on the Alpaca paper account in any session.
Common errors	Extended-hours order rejected online -> The asset or account may not be enabled for overnight trading or the limit is outside the price band; the stage falls back to a GTC limit that queues for the next session. Order stays 'new' or 'accepted' -> Normal outside regular hours or when the market is closed; it remains working - check the Orders page in the dashboard. insufficient qty available -> A stop or bracket leg reserves the shares; cancel_protective() must run before any sell. insufficient buying power -> Run with --reset offline, or lower the target weights.
Debrief prompt	Order types are tools for price, timing and risk, and the trading session decides which ones the broker accepts (K5). An order manager that reads the clock, prices limits off the quote, makes every order idempotent and prints its lifecycle is how an automated program knows exactly what happened at the broker (A3) - in New York's regular session or in its overnight session during a Singapore workday.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 9: Evaluation Stage: Performance and Risk Metrics with a Benchmark Table

Item	Facilitation
TSC / duration / slide	A3, K3 | 25 min | slide 136
Folder and files	labs_fullcode/Lab09-performance-metrics - Lab09-performance-metrics/evaluate_stage.py, pipeline/evaluate.py, common/metrics.py, Lab09-performance-metrics/solution/evaluate.py
Objective	Build `python -m pipeline evaluate` to compute total return, CAGR, volatility, Sharpe, Sortino, maximum drawdown, Calmar, win rate, profit factor, exposure, turnover, alpha and beta for every signal against a buy-and-hold benchmark, and interpret the ranking.
Run	Run Lab 09: python3 Lab09-performance-metrics/evaluate_stage.py (same as: python -m pipeline evaluate) The complete code: pipeline/evaluate.py (reference copy: Lab09-performance-metrics/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Read the metric formulas (K3) Build the evaluation stage (A3, K3) Run the evaluation (A3) Check one metric by hand (K3) Interpret trade and market metrics (A3, K3) Locate the worst periods (K3) Add a metric to the stage (K3)
Expected output	python -m pipeline evaluate --symbol SPY: window 2018-10-16 to 2025-12-31; ema_cross CAGR 11.09%, Sharpe 0.95, max drawdown -14.19% (best Sharpe); SPY buy & hold 15.28%, 0.81, -33.72%; trend_filter 9.70%, 0.81, -19.36%, 24 trades, win rate 41.67%, profit factor 3.79, beta 0.38; bollinger_breakout last (Sharpe 0.35); files _outputs/Lab09_metrics.csv and chart _outputs/Lab09_metrics.png.
Common errors	Sharpe looks too high -> Check that daily (not cumulative) returns are used and annualised with sqrt(252). Win rate below 50% yet profitable -> Expected for trend following: few large winners pay for many small losers (see profit factor). Strategies cover different periods -> Slice every return series from the same start date (first valid 200-day SMA).
Debrief prompt	Performance metrics (K3) answer different questions: CAGR measures growth, Sharpe and Sortino reward per unit of risk, drawdown and Calmar the pain an investor must survive, beta how much is just the market. Running every strategy through one evaluation stage (A3) is what makes the comparison fair.

Topic 3: Machine Learning Models, Statistical Modelling and A/B Testing for Trading Signals

Day 2 | Slides 141-169 | A4, A5, K2, K4, K6 | Labs 10-14

Learning outcome: Conduct statistical modelling of market data with machine learning models to derive patterns and solutions, and perform A/B testing of trading strategies based on different hypotheses.

Key teaching points

- 3.1 Types of Statistical Analyses, Data Models, Algorithms and Computational Methods (K2, A2): Descriptive analysis summarises what happened (mean return, volatility, skew, correlation); inferential analysis tests whether an effect is real (t-test, bootstrap); predictive analysis forecasts (regression, classification); prescriptive analysis decides (optimisation).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- 3.2 Supervised Learning Models for Trading Signals (K4, A4): Target and features: the target is what you predict (next-day direction); the features are what you know at prediction time (lagged returns, RSI, MACD, volatility, volume change). Any feature using future data is leakage.
- 3.3 Unsupervised Learning: Clustering and PCA for Market Regimes (K4, A4): K-means clustering partitions observations into k groups by minimising within-cluster variance. Applied to rolling return and volatility features it separates calm-bull, choppy and stressed market regimes.
- 3.4 Statistical Modelling Techniques for Financial Time Series (K6, A4): Returns, not prices: model log or simple returns, which are closer to stationary. Prices trend and wander; regressions on raw prices produce spurious results.
- 3.5 A/B Testing Trading Strategies (A5, K2): An A/B test compares variant A (for example a 20/50 crossover) with variant B (a 10/30 crossover, or the same rule with an RSI filter) on the same data and period, to decide whether B is genuinely better.
- 3.6 Pairs Trading: Statistical Arbitrage as a Hedging Algorithm (K6, A4): Pairs trading holds one stock long and an economically linked stock short, in hedge-ratio proportion. Market moves that hit both legs largely cancel, so the position is a hedge: its profit or loss comes from the spread between the two, not from the direction of the market.

Facilitation cues

- Show the strategy-search heatmap from Lab 10 and ask: 'Which cell would you trade?' Then show out-of-sample decay - the best in-sample cell is rarely best afterwards.
- In Lab 11, accuracy near 50-55% is normal for daily direction; the point is the net-of-cost strategy, not the accuracy.
- For the A/B test, make learners write H0, H1 and alpha BEFORE running Lab 13. This is a CRITICAL marking point in PP Task 3.
- In Lab 14 show that JPM/BAC is 93% correlated yet NOT cointegrated, and that the KO/PEP hedge leaves almost no market beta even though the pair lost money after 2022 - the hedge works, the edge did not persist.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 10: Strategy-Search Stage: Grid Search, Out-of-Sample Ranking and Decay

Item	Facilitation
TSC / duration / slide	A2, A4, K2 | 20 min | slide 159
Folder and files	labs_fullcode/Lab10-strategy-grid-search - Lab10-strategy-grid-search/search_stage.py, pipeline/search.py, Lab10-strategy-grid-search/solution/search.py
Objective	Build `python -m pipeline search` to grid-search three signal families, rank parameter sets in sample, measure out-of-sample decay, prefer a robust parameter region, and decide whether anything deserves promotion into config.yaml.
Run	Run Lab 10: python3 Lab10-strategy-grid-search/search_stage.py (same as: python -m pipeline search) The complete code: pipeline/search.py (reference copy: Lab10-strategy-grid-search/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Define the search space (A2, K2) Build the grid search (A2, A4) Add ranking, robustness and the decay report (A4, K2) Run the search (A4) Read the overfitting evidence (A4, K2) Prefer a robust region to a single peak (A2, A4) Decide whether to promote anything (A2, K2)
Expected output	python -m pipeline search --symbol SPY: 76 parameter sets; buy-and-hold Sharpe 0.87 in sample and 0.67 out of sample; best sma_cross (10/100) 1.23 -> 0.70; best rsi_reversion (2, 30, 70) 0.60 -> 0.74; best bollinger_breakout (20, 1.0) 0.91 -> 1.04; top-5 average decay -0.62; IS/OOS rank correlation 0.06; 43% of sets beat buy and hold out of sample; files _outputs/Lab10_grid_results.csv, _outputs/Lab10_decay_report.csv and chart _outputs/Lab10_grid_heatmap.png.
Common errors	IntCastingNaNError on ranks -> Some sets never trade in one window (NaN Sharpe); rank with na_option='bottom'. Heatmap has a blank cell -> Expected: fast 50 / slow 50 is not a valid pair. Different results online -> Alpaca's IEX data differ slightly from the bundled data; compare patterns, not decimals.
Debrief prompt	Grid search is a statistical search over model structures (A2, A4); comparing in-sample with out-of-sample results (K2) exposes how much of the 'best' result was luck. A hold-out period, a count of trials and a preference for robust regions are the pipeline's defences against fooling itself.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 11: ML-Signal Stage: a Walk-Forward Classifier from Probability to Position

Item	Facilitation
TSC / duration / slide	A4, K4, K6 | 20 min | slide 161
Folder and files	labs_fullcode/Lab11-ml-signal-classifier - Lab11-ml-signal-classifier/ml_stage.py, pipeline/ml.py, Lab11-ml-signal-classifier/solution/ml.py
Objective	Build pipeline/ml.py: engineer causal features, train logistic regression, random forest and XGBoost with walk-forward TimeSeriesSplit validation, turn P(up) into a target position and backtest it after costs.
Run	Run Lab 11: python3 Lab11-ml-signal-classifier/ml_stage.py (same as: python -m pipeline ml) The complete code: pipeline/ml.py (reference copy: Lab11-ml-signal-classifier/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Build causal features and the label (K4) Add walk-forward validation (K4, K6) Turn probability into a position and backtest it (A4) Run the ML stage (A4, K4) Read the strategy result (A4) Test the threshold - and notice the snooping risk (A4, K6) Prove that leakage is detectable (K6) Inspect the feature importance (K4)
Expected output	python -m pipeline ml --threshold 0.55: 1,811 samples, 14 features, base rate 55.4%; out-of-sample AUC 0.481 (logistic regression), 0.483 (random forest), 0.497 (XGBoost); strategies 2020-2025: CAGR 9.19% / 7.70% / 4.00% versus buy and hold 14.98%, Sharpe 0.61 / 0.54 / 0.32 versus 0.78, turnover 75-96 versus 0.17; VERDICT: NO EDGE; a leaky feature pushes AUC to 0.997; files _outputs/Lab11_ml_probabilities.csv and chart _outputs/Lab11_ml_strategy.png.
Common errors	xgboost not installed -> Run pip install xgboost, or use --models 'Logistic regression' 'Random forest'. Accuracy above 60% -> Suspect leakage: no feature may use shift(-1), and TimeSeriesSplit needs gap=1. Results differ in the third decimal -> Random forest and XGBoost use random_state=42; library versions can still differ slightly.
Debrief prompt	Supervised models (K4) can be trained and validated correctly (K6) and still find no edge - an honest negative result is valuable evidence (A4). Walk-forward validation stops the model learning from the future, a leakage test proves the check works, and the cost-aware backtest shows what an AUC of 0.5 is worth in money.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 12: Regime-Filter Stage: PCA and K-Means as a Gate on the Signals

Item	Facilitation
TSC / duration / slide	A4, K4, K6 | 15 min | slide 163
Folder and files	labs_fullcode/Lab12-market-regimes - Lab12-market-regimes/regime_stage.py, pipeline/regime.py, Lab12-market-regimes/solution/regime.py
Objective	Build pipeline/regime.py: PCA on the universe's returns, K-means regimes on backward-looking SPY features, walk-forward regime labels, and a gate that forces signals flat in the stress regime, evaluated after costs.
Run	Run Lab 12: python3 Lab12-market-regimes/regime_stage.py (same as: python -m pipeline regime) The complete code: pipeline/regime.py (reference copy: Lab12-market-regimes/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Build PCA and the regime features (K4) Add walk-forward labels and the gate (K6, A4) Run the regime stage and read the PCA (A4, K4) Read the regime profiles (K4, K6) Check the walk-forward labels (K6) Evaluate the gate after costs (A4) Try four clusters (A4)
Expected output	python -m pipeline regime --clusters 3: PC1 56.8%, PC2 15.5%, PC3 10.3% (82.5% cumulative); regimes Low vol (654 days, vol 12.7%), Moderate vol (786, 13.7%), High vol (stress) (372, 30.4%); trend-filter Sharpe 1.23 / 1.04 / -0.45 by regime; walk-forward 2020-2025: 1,061 / 383 / 64 days, latest Low vol (gate open); gate effect on Sharpe: ema_cross 0.91 -> 0.89, trend_filter 0.89 -> 0.80, dip_buy 0.63 -> 0.72; files _outputs/Lab12_regime_labels.csv and chart _outputs/Lab12_regimes.png.
Common errors	Cluster names swap between runs -> K-means numbers clusters arbitrarily; fit_regimes names them by volatility so labels stay meaningful. ConvergenceWarning -> Increase n_init or max_iter in KMeans; results are usually unaffected. Walk-forward starts in 2020 -> The first fit needs at least 200 days of features after the 200-day warm-up.
Debrief prompt	Unsupervised learning (K4) finds structure without labels: PCA compresses correlated assets into a few drivers and K-means groups days into regimes. Fitting the regimes walk-forward is part of validating the model (K6), and testing the gate signal by signal after costs (A4) shows that a filter can help one strategy and hurt another.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 13: A/B-Test Stage: a Statistical Decision before Capital Moves

Item	Facilitation
TSC / duration / slide	A5, K2 | 15 min | slide 165
Folder and files	labs_fullcode/Lab13-ab-testing - Lab13-ab-testing/abtest_stage.py, pipeline/abtest.py, Lab13-ab-testing/solution/abtest.py
Objective	Build `python -m pipeline abtest` to compare any two library signals with a Welch t-test, a paired t-test and a block-bootstrap confidence interval for the Sharpe difference, and to record an accept or reject decision with a multiple-testing caution.
Run	Run Lab 13: python3 Lab13-ab-testing/abtest_stage.py (same as: python -m pipeline abtest) The complete code: pipeline/abtest.py (reference copy: Lab13-ab-testing/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	State the hypotheses (A5, K2) Build the statistics (A5, K2) Build the A/B command (A5) Run the A/B test (A5) Compare the independent and paired tests (K2) Read the bootstrap CI and the decision (A5) Test the pipeline's own strategy (A5, K2)
Expected output	python -m pipeline abtest --a trend_filter --b dip_buy: window 2018-10-16 to 2025-12-31 (1,812 days); CAGR 9.70% vs 4.97%, Sharpe 0.81 vs 0.49; Welch t = 0.74 (p = 0.462); paired t = 0.85 (p = 0.398); Sharpe difference +0.31, 95% CI [-0.33, +1.05]; A beats B in 82% of bootstrap samples; DECISION: do not reject H0; Bonferroni p < 0.0025 for 20 variants; files _outputs/Lab13_abtest_decision.yaml and chart _outputs/Lab13_ab_test.png.
Common errors	p-value differs slightly -> The bootstrap depends on the seed (7) and the number of draws; the decision should not change. t-tests and the CI disagree -> The t-tests compare mean returns, the CI compares Sharpe ratios; report both. Returns are not normal -> Daily returns have fat tails - that is why the block bootstrap is used alongside the t-tests.
Debrief prompt	A/B testing (A5) applies statistical inference to a business choice: the trend filter looks better, but the evidence cannot reject 'no difference'. Statistical tests (K2) - paired where the data are paired, bootstrapped where returns are not normal, corrected for the number of variants tried - protect capital from being moved on noise.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 14: Pairs Stage: a Hedged, Market-Neutral Statistical-Arbitrage Sleeve

Item	Facilitation
TSC / duration / slide	A4, K6 | 20 min | slide 167
Folder and files	labs_fullcode/Lab14-pairs-trading - Lab14-pairs-trading/pairs_stage.py, pipeline/pairs.py, data/KO.csv, data/PEP.csv, data/BAC.csv, Lab14-pairs-trading/solution/pairs.py
Objective	Build pipeline/pairs.py: screen pairs with the Engle-Granger cointegration test, estimate the hedge ratio, trade the spread z-score out of sample with next-bar execution and costs on both legs, confirm market neutrality and output leg targets.
Run	Run Lab 14: python3 Lab14-pairs-trading/pairs_stage.py (same as: python -m pipeline pairs) The complete code: pipeline/pairs.py (reference copy: Lab14-pairs-trading/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Read the method (K6) Build the cointegration tools (K6) Build the spread trading rules and leg targets (A4) Run the pair screen (A4, K6) Interpret the hedge ratio (K6) Evaluate the out-of-sample result (A4) Test a failed pair and other thresholds (A4, K6) Study the chart (A4)
Expected output	python -m pipeline pairs: screening KO/PEP ADF t -3.36 (cointegrated at 5%), JPM/BAC -2.08 and XLF/JPM -1.89 (not); KO/PEP hedge ratio 0.756, half-life 39 days; out of sample 2023-2025: 13 trades, 59% time in market, gross -9.4%, net -10.6% (Sharpe -0.60), beta to SPY +0.02, versus SPY +86.3%; leg targets KO 0.0 / PEP 0.0 at 2025-12-31; chart _outputs/Lab14_pairs_trading.png.
Common errors	KeyError for KO, PEP or BAC -> The bundled data folder is out of date; copy data/KO.csv, data/PEP.csv and data/BAC.csv from the course lab folder. ADF t differs slightly online -> Alpaca's IEX feed and the bundled Yahoo data differ a little; use the bundled data for the class comparison. All pairs show 'no' -> Check the formation period is 2018-2022 and that log prices are used.
Debrief prompt	Pairs trading is a hedging algorithm: the short leg cancels most of the market exposure of the long leg, which is why the beta to SPY is near zero (A4). The statistical model is cointegration, not correlation (K6). A pair that is cointegrated in one period can stop reverting in the next, so professional desks re-test cointegration on a rolling basis and size each pair small.

Topic 4: Building an End-to-End Algorithmic Trading Pipeline: Backtest, Forward Test and Performance Analysis

Day 2 | Slides 170-193 | A7, A8, A9, A10, K7, K8 | Labs 15-20

Learning outcome: Perform coding and configuration of an automated trading program based on selected models or algorithms, conduct tests on its actions and outcomes, diagnose unintended outcomes, and draw insights from the results to support decisions and report accordingly.

Key teaching points

- 4.1 Architecture of an End-to-End Algorithmic Trading Pipeline (A7, K7): The pipeline: (1) strategy search and research, (2) data ingestion, (3) signal generation, (4) backtest with costs, (5) risk checks, (6) order execution, (7) forward test on paper, (8) monitoring, logging and performance reporting.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- 4.2 Backtesting with Realistic Trading Costs (A8, K7): A backtest replays a strategy on historical data. Vectorised backtests are fast (whole-array operations); event-driven backtests replay bar by bar and model orders more realistically.
- 4.3 Forward Testing with Automated Paper Trading (A7, A8): Forward testing (paper trading) runs the strategy in real time on data it has never seen, placing simulated orders - the bridge between a backtest and live capital.
- 4.4 Potential Reasons for Unintended Outcomes and How to Diagnose Them (A9, K8): Overfitting: parameters tuned to noise. Signs - a spectacular in-sample Sharpe that collapses out of sample, and performance that is very sensitive to small parameter changes.
- 4.5 Performance Analysis, Insights and Reporting (A10): A tearsheet presents the equity curve against the benchmark, the drawdown chart, monthly returns, rolling Sharpe ratio, the metrics table and the cost breakdown on one page.

Facilitation cues

- Draw the pipeline on the whiteboard first; every Topic 4 lab is one box of it.
- In Lab 16, run the bot once together in offline replay, then let learners with keys run it against their paper account during market hours if the class time overlaps US trading hours (21:30-04:00 SGT, 22:30-05:00 in northern winter).
- Lab 18 contains seeded faults: do not reveal them. Prompt with 'Is this result too good?' and 'On which bar was this order actually filled?'
- In Lab 20 stress that the LLM never bypasses the coded risk manager, and that every decision is logged - the governance message of Topic 1, implemented.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 15: Backtest-Engine Stage: Cost Models, Execution Timing and Break-Even

Item	Facilitation
TSC / duration / slide	A7, A8, K7 | 25 min | slide 182
Folder and files	labs_fullcode/Lab15-trading-costs - Lab15-trading-costs/backtest_stage.py, pipeline/backtest.py, common/backtest.py, Lab15-trading-costs/solution/backtest.py
Objective	Build pipeline/backtest.py around the common backtester: run any library signal under every cost scenario (and a custom one), break the cost into commission, spread, slippage and regulatory fees, find each strategy's break-even cost and test next-open versus next-close execution.
Run	Run Lab 15: python3 Lab15-trading-costs/backtest_stage.py (same as: python -m pipeline backtest) The complete code: pipeline/backtest.py (reference copy: Lab15-trading-costs/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Read the cost model (K7) Build the engine helpers (A7) Build the scenario report (A7, A8) Run the cost scenarios (A7, A8) Analyse the cost breakdown (K7) Interpret break-even and execution timing (A8, K7) Configure your own execution route (A7)
Expected output	python -m pipeline backtest: dip_buy gross CAGR 5.09%, net 4.65% (Alpaca) / 4.21% (per-share) / 0.47% (high-cost, USD 33,560 of costs); trend_filter gross 8.80%, net 8.70% / 8.59% / 7.68%; break-even 19.5 bps per side for dip_buy (102 round trips, average trade 41.7 bps) versus 142.3 bps for trend_filter; next-close fills lift dip_buy to 7.22%; custom route 3.89% / 8.51%; files _outputs/Lab15_cost_scenarios.csv and chart _outputs/Lab15_costs.png.
Common errors	Net CAGR equals gross CAGR -> You are using the zero preset; check costs.preset and costs.scenarios in config.yaml. Regulatory fees look tiny -> They are: a few dollars per million of sell notional. Rates change - check the current SEC and FINRA rates. Break-even hits the upper bound -> Increase hi in break_even_bps() for very slow strategies.
Debrief prompt	Commission is only one trading cost (K7): spread and slippage scale with turnover, and the assumed fill time can matter as much as the fee schedule. Coding the cost model into the engine (A7) and testing outcomes under several scenarios (A8) shows whether an edge survives real-world execution - the break-even cost is the single number that says how much room the strategy has.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 16: Automation Stage: a Scheduled, Risk-Checked Forward Test and Live Paper Trading

Item	Facilitation
TSC / duration / slide	A7, A8, K7 | 25 min | slide 184
Folder and files	labs_fullcode/Lab16-paper-trading-bot - Lab16-paper-trading-bot/run_stage.py, pipeline/runner.py, pipeline/risk.py, Lab16-paper-trading-bot/solution/runner.py, Lab16-paper-trading-bot/solution/risk.py
Objective	Build pipeline/risk.py and pipeline/runner.py: `python -m pipeline run --once | --loop` with the market clock, risk checks, kill switch and trade log; run a 2025 forward test; run the strategy live on the paper account for a few cycles; and schedule it with cron, launchd or Windows Task Scheduler.
Run	Run Lab 16: python3 Lab16-paper-trading-bot/run_stage.py (same as: python -m pipeline run) The complete code: pipeline/risk.py, pipeline/runner.py (reference copy: Lab16-paper-trading-bot/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Build the risk layer and the kill switch (A7) Build one decision cycle (A7) Build the forward-test replay and the logs (A8) Add the live paper loop and scheduling (A7) Run the 2025 forward test (A8, K7) Audit the decision log (A8) Test the kill switch (A7) Run the strategy LIVE on the paper account (A7, A8, K7) Schedule the runner (A7)
Expected output	python -m pipeline run --loop (offline 2025 replay): 250 sessions, 5 orders (BUY 159, SELL, BUY 166, SELL, BUY 163), equity USD 100,000.00 -> 116,249.79 (+16.25%) versus SPY +17.06%; trade_log.csv with 250 rows; kill switch halts the next cycle and is archived on --off; run --loop --live-paper sends a 10% SPY order as an extended-hours limit during Singapore hours and stops on the kill switch; schedule entries for 09:35 New York (21:35 / 22:35 Singapore); chart _outputs/Lab16_forward_test.png.
Common errors	Action BLOCKED with 'market closed' -> Expected for --once at a weekend online; the live loop queues a GTC order instead (allow_queue). No trades in the replay -> Check that labs_fullcode/KILL_SWITCH does not exist (python -m pipeline kill --status). A cron job ran twice -> The client_order_id is unique per decision bar, so the second run finds the order and sends nothing. macOS cron fires at the wrong hour -> macOS cron ignores CRON_TZ; use the Singapore-time line or launchd.
Debrief prompt	The runner is a software agent coded from a selected model (A7): signal on completed bars, deterministic risk limits, idempotent execution, a kill switch and an audit log. The forward test and the live paper loop check the actions it takes and their outcomes on unseen data (A8), including the real effect of whole shares, fills and session rules (K7).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 17: Event-Driven Automation: TradingView Webhook to Signal, Risk and Execution

Item	Facilitation
TSC / duration / slide	A7, K5, K7 | 20 min | slide 186
Folder and files	labs_fullcode/Lab17-tradingview-webhook - Lab17-tradingview-webhook/webhook_stage.py, Lab17-tradingview-webhook/send_test_alert.py, Lab17-tradingview-webhook/lcc_webhook_strategy.pine, Lab17-tradingview-webhook/tradingview_alert_message.json, pipeline/webhook.py, Lab17-tradingview-webhook/solution/webhook.py
Objective	Build pipeline/webhook.py: a webhook receiver that turns TradingView alert JSON into a pipeline decision (validation -> signal -> risk -> execution), test it with a local alert simulator and a self-test, and configure a real TradingView strategy alert.
Run	Run Lab 17: python3 Lab17-tradingview-webhook/webhook_stage.py (same as: python -m pipeline webhook) Also in this lab: send_test_alert.py: python3 Lab17-tradingview-webhook/send_test_alert.py The complete code: pipeline/webhook.py (reference copy: Lab17-tradingview-webhook/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Set the shared secret (A7) Build alert validation (A7, K5) Route valid alerts through signal, risk and execution (A7, K7) Build the server, the simulator and the self-test (A7) Run the self-test (A7, K5) Run the receiver and the simulator in two terminals (A7) Let the pipeline's signal veto an alert (A7, K7) Prove the kill switch reaches the webhook (A7) Optional - connect a real TradingView alert (paid plan) (A7, K5)
Expected output	python -m pipeline webhook --selftest: HTTP 200 (BUY 5 SPY filled at 676.70), 200 (SELL 5 SPY), 401 'bad or missing secret', 422 'TSLA not on the allow-list', 422 'qty must be between 0 and 100', 400 'body is not valid JSON', 200 'duplicate ignored'; RESULT: 7/7; with require_signal_agreement the SELL returns 409; with the kill switch on, the BUY returns 423; audit log _outputs/Lab17_webhook_log.csv.
Common errors	Address already in use -> Another program uses port 8080; start with --port 8081 and use the same port in --url. All alerts return 401 -> The receiver and sender use different secrets; both read WEBHOOK_SECRET from labs_fullcode/.env, or pass --secret to the simulator. TradingView shows 'webhook delivery failed' -> Use the HTTPS tunnel URL on port 443, keep the receiver running and check the alert message is valid JSON. Alerts return 423 -> The kill switch is on or the market is closed online; check python -m pipeline kill --status.
Debrief prompt	A webhook lets a charting platform trigger execution on a trading platform (K5), but the gateway code (A7) is where the controls live: authentication, allow-lists, size limits, duplicate protection, the pipeline's own signal, the risk layer and an audit log. Each automated order still pays spread and slippage (K7), so alert frequency matters.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 18: Diagnostics Stage: Automated Bias Checks before Go-Live

Item	Facilitation
TSC / duration / slide	A9, K8 | 20 min | slide 187
Folder and files	labs_fullcode/Lab18-diagnose-bias - Lab18-diagnose-bias/diagnose_stage.py, Lab18-diagnose-bias/buggy_backtest.py, pipeline/diagnose.py, Lab18-diagnose-bias/solution/diagnose.py
Objective	Build `python -m pipeline diagnose` to run automated look-ahead (features and execution), cost-drag, overfitting and survivorship checks on any strategy and on the junior quant's exhibit, and write a PASS / WARN / FAIL report with a go-live verdict.
Run	Run Lab 18: python3 Lab18-diagnose-bias/diagnose_stage.py (same as: python -m pipeline diagnose) Also in this lab: buggy_backtest.py: python3 Lab18-diagnose-bias/buggy_backtest.py The complete code: pipeline/diagnose.py (reference copy: Lab18-diagnose-bias/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Run the suspicious backtest (A9) Find the problems (A9, K8) Build the look-ahead checks (A9, K8) Build the cost, overfitting and survivorship checks (A9, K8) Run the diagnostics (A9) Read the exhibit fixed one problem at a time (A9, K8) Read the overfitting and survivorship evidence (K8) Use the stage as a gate and write the committee note (A9)
Expected output	python -m pipeline diagnose: ema_cross PASS for look-ahead (0 of 25), execution (same-bar 1.45 vs next-bar 0.82) and cost drag (break-even 135.8 bps vs 1.5 bps), WARN for overfitting (IS 0.86 vs best of 76 random 1.42) and survivorship -> CLEAR for paper trading; exhibit FAIL: 25 of 25 look-ahead checks, Sharpe 4.28 as submitted -> 0.24 fixed -> 0.18 with costs; 500 random strategies best IS 1.77 -> OOS 0.55; AI basket 38.84% vs SPY 14.15%; files _outputs/Lab18_diagnostics.csv and charts _outputs/Lab18_bias_comparison.png and _outputs/Lab18_overfitting.png.
Common errors	Step 1 shows a slightly different CAGR from the fix table -> buggy_backtest.py drops the first NaN rows; the stage fills them with 0 - 71.52% vs 71.47%. Random-strategy numbers differ -> Change the seed or --trials and the threshold changes, but the pattern (no persistence) stays. Is center=True really look-ahead? -> Yes: a centred 5-day mean at day t uses the closes of t+1 and t+2.
Debrief prompt	Most 'too good to be true' results come from a short list of causes (K8): look-ahead in features or execution, overfitting, survivorship and ignored costs. Automating the checks and using the verdict as a gate (A9) is what stops an unintended outcome from reaching even a paper account - and the same checks run on the desk's own strategy, not only on other people's.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 19: Monitoring and Reporting Stage: Automated Tearsheet and Backtest-vs-Forward Report

Item	Facilitation
TSC / duration / slide	A8, A10 | 25 min | slide 189
Folder and files	labs_fullcode/Lab19-performance-report - Lab19-performance-report/report_stage.py, pipeline/report.py, Lab19-performance-report/solution/report.py
Objective	Build `python -m pipeline report` to compare the research backtest with the runner's forward-test logs, run monitoring checks with thresholds, and generate a tearsheet and a markdown report with data-driven insights and a recommendation.
Run	Run Lab 19: python3 Lab19-performance-report/report_stage.py (same as: python -m pipeline report) The complete code: pipeline/report.py (reference copy: Lab19-performance-report/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Make sure the forward-test logs exist (A8) Build the backtest-vs-forward comparison (A8) Add monitoring checks, the tearsheet and the report (A8, A10) Generate the report (A8, A10) Read the monitoring checks (A8) Trigger an alert on purpose (A8) Study the tearsheet (A10) Write the committee recommendation (A10)
Expected output	python -m pipeline report: 2018-2025 strategy CAGR 9.02% vs SPY 14.15%, Sharpe 0.81 vs 0.78, max drawdown -14.19% vs -33.72%, beta 0.35, costs USD 1,072 (drag 0.11% a year); 2025 backtest +17.04% (Sharpe 1.64) vs forward test +16.25% (Sharpe 1.65), 5 orders each, average slippage 1.0 bps, tracking -0.79%; 5/5 monitoring checks OK; recommendation: continue paper trading; files _outputs/Lab19_tearsheet.png, _outputs/Lab19_report.md and _outputs/Lab19_monitoring.csv.
Common errors	ImportError: tabulate -> Run pip install tabulate (needed for the markdown tables). Diagnostics check shows ALERT 'not run' -> Run python -m pipeline diagnose for the configured strategy first. Forward-test numbers differ -> Rerun python -m pipeline run --loop with the default dates; --once rows outside 2025 are ignored.
Debrief prompt	Comparing the backtest with the forward test is the final test of whether actions and outcomes match the model (A8). Automated monitoring with explicit thresholds turns that comparison into an alarm, and a good report turns numbers into trends, insights and a clear, conditional recommendation (A10) - including the honest message that the strategy reduces risk but has not beaten buy and hold.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab 20: Capstone: One Command Runs the Pipeline End to End, with an Agentic Layer and Human Approval

Item	Facilitation
TSC / duration / slide	A7, A10, K1 | 20 min | slide 191
Folder and files	labs_fullcode/Lab20-agentic-capstone - Lab20-agentic-capstone/e2e_stage.py, pipeline/e2e.py, pipeline/agents.py, Lab20-agentic-capstone/solution/e2e.py, Lab20-agentic-capstone/solution/agents.py
Objective	Build pipeline/agents.py (analyst, researcher, trader and risk agents; rule-based by default, Claude optional) and pipeline/e2e.py (`python -m pipeline e2e`) that runs every stage in order with gates, then evaluate the pipeline and the role of human oversight.
Run	Run Lab 20: python3 Lab20-agentic-capstone/e2e_stage.py (same as: python -m pipeline e2e) The complete code: pipeline/agents.py, pipeline/e2e.py (reference copy: Lab20-agentic-capstone/solution/) Outputs: _outputs/ (charts, CSV files and reports written by the stage)
Python steps (summary)	Build the agent layer (K1, A7) Build the end-to-end orchestrator (A7, A10) Run the whole pipeline (A7) Trace the run (A10) Read the agents' decisions (K1, A10) Apply human oversight (A7, K1) Prove the gates stop the orders (A7) Optional - switch on the LLM agents (K1) Run the capstone on the live paper account (A7) Evaluate the pipeline (A10, K1)
Expected output	python -m pipeline e2e (offline): 20 stages OK in about 6-10 seconds, 0 errors, gates passed - from check and validation through the 2025 forward test (+16.25%), webhook self-test (7/7), diagnostics (CLEAR) and report (5/5 monitoring OK) to the agents (NVDA BUY 107 with stop 175.5 / target 201.9, SPY BUY 22 with 664.75 / 694.45, AAPL HOLD); with --execute and YES, 2 paper bracket orders; with the kill switch on, 0 orders; files _outputs/Lab20_e2e_summary.json, _outputs/Lab20_e2e_log.txt, _outputs/Lab20_decision_trace.json and charts _outputs/Lab20_e2e_pipeline.png and _outputs/Lab20_agent_pipeline.png.
Common errors	A stage shows ERROR -> Open _outputs/Lab20_e2e_log.txt - the traceback is under that stage; fix it and rerun (python -m pipeline <stage> runs it alone). Brain shows rule-based although a key is set -> Install the anthropic package and check ANTHROPIC_API_KEY in labs_fullcode/.env; failures fall back to rules by design. All agent decisions are HOLD on a second run -> You already hold the positions; agents run with --reset in e2e, or add --reset when running agents alone. Headlines look unfamiliar -> Offline headlines are SYNTHETIC samples for training, not real news; online the Alpaca News API is used.
Debrief prompt	Agentic AI (K1) chains specialised agents that reason, debate and act, which makes it powerful and risky. Wiring it into an end-to-end pipeline whose gates, deterministic risk limits, kill switch and human approval it cannot override (A7), and evaluating every stage's evidence critically (A10), is how a desk adopts AI and automation responsibly.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

5. Assessment Administration

The Assessment Plan is the governing document for the assessment; this section summarises what the trainer-assessor does on the day.

Component	Duration	Coverage	Conditions	Ratio
Written Assessment (Short Answer Questions)	45 min	9 questions, K1-K9	Open book, individual, invigilated	1 : 20
Practical Performance	75 min	4 tasks, A1-A10	Open book, individual, own laptop with Python; Alpaca paper API or the supplied simulator	1 : 20

Sequence on the day

1. Verify each candidate's identity; candidates sign the assessment attendance record.
2. Brief candidates (5 minutes, not assessed): components, durations, open book, individual work, pass rule, appeal and re-assessment.
3. Issue the Written Assessment question paper (Assessment Plan Annex A); start the 45-minute clock; collect scripts at time.
4. Copy courseware/assessment/pp_assessment/candidate into each candidate's labs_fullcode/pp_assessment/ (never the assessor folder), then issue the Practical Performance question paper (Annex C); start the 75-minute clock. Observe and note process evidence.
5. Collect the candidate's script, outputs, order log and completed response template (saved as PP_<Name>_<Date>).
6. Mark against the marking points (Annex B / D / E), apply the Decision Rules and the Assessor Key (Annex F), derive bands, complete the Score Summary and the Individual Assessment Records.
7. Give feedback. A candidate who is Not Yet Competent is counselled on the gaps and offered re-assessment on the component(s) not met.

Pass rule

- (1) Written Assessment: at least 80 of 100 marks.
- (2) Practical Performance: at least 80 of 100 marks.
- (3) Every knowledge statement (K1-K9) and every ability statement (A1-A10) scores at least 60% of the marks allocated to it (table below), so that each statement is demonstrated.
- (4) Every CRITICAL marking point (1.1, 3.5, 4.1, 4.5, 4.9) is awarded.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

6. Assessor's Keys

The assessor's keys, expected values and the marking rubric are trainer-only and are held in the full Facilitator Guide and the Assessment Plan.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.